VerdeScope • Lab

VERDESCOPE · DOCUMENTACIÓN

Arquitectura y tecnologías

Arquitectura hexagonal, flujo de datos, stack y decisiones

Versión 0.2
Fecha 25 de octubre de 2026
Autoría Equipo VerdeScope · proyecto independiente
Estado Documento iterativo · experimento personal

Contenido

1 Visión general **2**
2 Arquitectura hexagonal **3**
3 Diagrama de componentes **4**
4 Flujo de datos **5**
5 Tecnologías **6**
5.1 Stack de la demo y objetivo de producción 6
5.2 Justificación de elecciones clave 6
6 Decisiones de arquitectura (ADR) **7**
7 Principios Clean Code aplicados **8**
8 Estrategia de testing **9**

1 Visión general

VerdeScope separa con claridad el **dominio** (reglas de negocio puras), los **casos de uso**, la **infraestructura** (datos, modelos, persistencia) y las **interfaces** (API FastAPI y web estática). El resultado es un sistema que se despliega de dos formas con el mismo código de dominio: como **sitio estático** (GitHub Pages) y como **servicio corporativo** con API y base de datos.

2 Arquitectura hexagonal

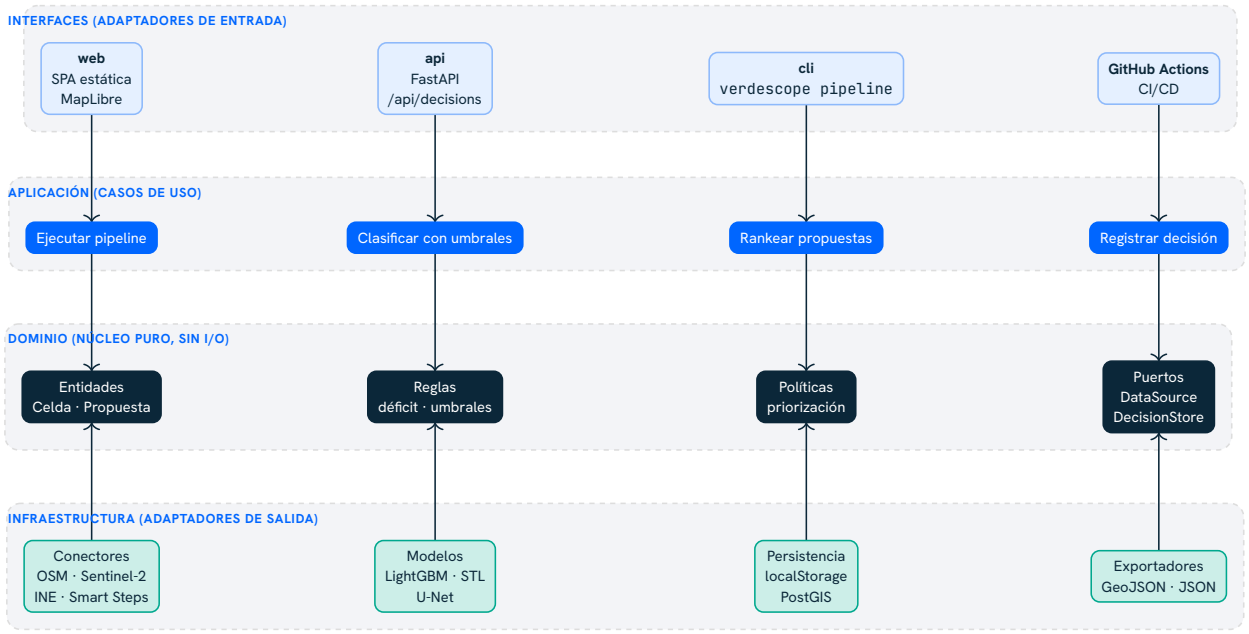


Figura 1: Capas domain, application, infrastructure, api y web. La regla de dependencia apunta siempre hacia el dominio.

DECISIÓN

Regla de dependencia. `domain` no importa nada de las demás capas ni de librerías de E/S. `application` sólo conoce los puertos del dominio. `infrastructure` y `api` implementan o consumen esos puertos. Esto permite sustituir una fuente sintética por Sentinel-2 real o el almacenamiento local por PostGIS sin tocar reglas de negocio.

Capa	Responsabilidad y contenido (backend/src/verdescope/)
`domain`	Entidades (celda, propuesta, decisión), reglas puras (clasificación por umbrales, normalización), políticas de priorización. Probable de probar sin mocks.
`application`	Casos de uso: ejecutar el pipeline, calcular propuestas, registrar decisiones. Orquesta puertos.
`infrastructure`	Conectores `DataSource` (OSM/Overpass, STAC, INE, Smart Steps), modelos (LightGBM, SHAP, STL, Theil-Sen), exportadores de artefactos.
`api`	FastAPI: `GET/POST /api/decisions` ; capa opcional para producción.
`web`	SPA estática en `docs/` : lee los JSON del pipeline, aplica umbrales en el cliente y guarda decisiones mediante `DecisionStore` .

3 Diagrama de componentes

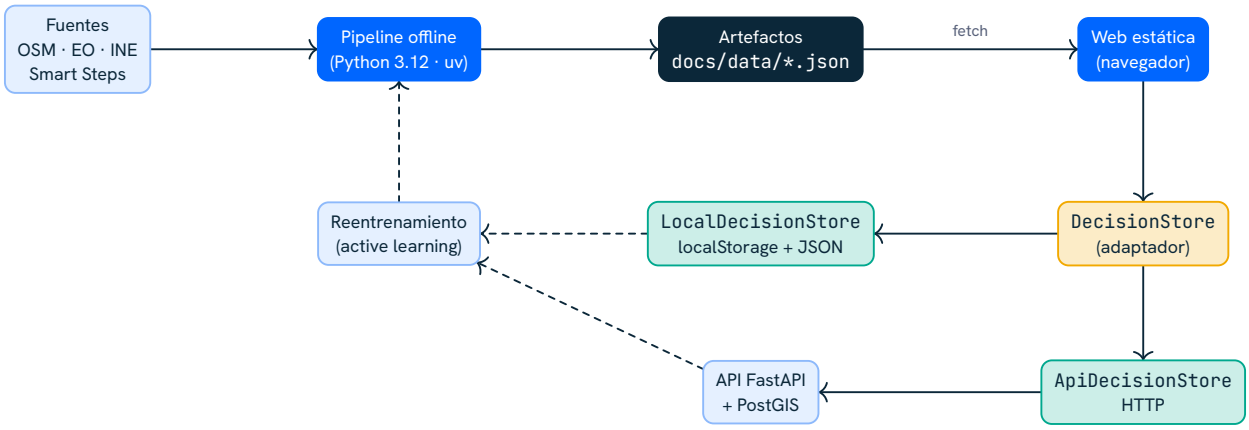


Figura 2: Componentes y adaptadores. La web elige el adaptador según `config.json` (`api_base_url`: `null` → local).

4 Flujo de datos

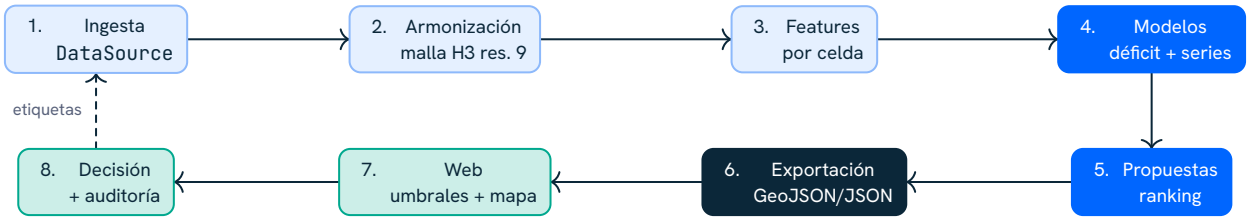


Figura 3: De la ingesta a la decisión: el bucle se cierra con las decisiones como etiquetas.

Los artefactos siguen un contrato de datos versionado (`schema_version` en `manifest.json`): `cells.geojson` (hexágonos con `deficit_score`, `uncertainty`, `top_factors`…), `proposals.geojson`, `timeseries.json` y `green_areas.geojson`. La clasificación crítico/revisar/ok **no se persiste**: se calcula en el cliente con los umbrales vigentes.

5 Tecnologías

5.1 Stack de la demo y objetivo de producción

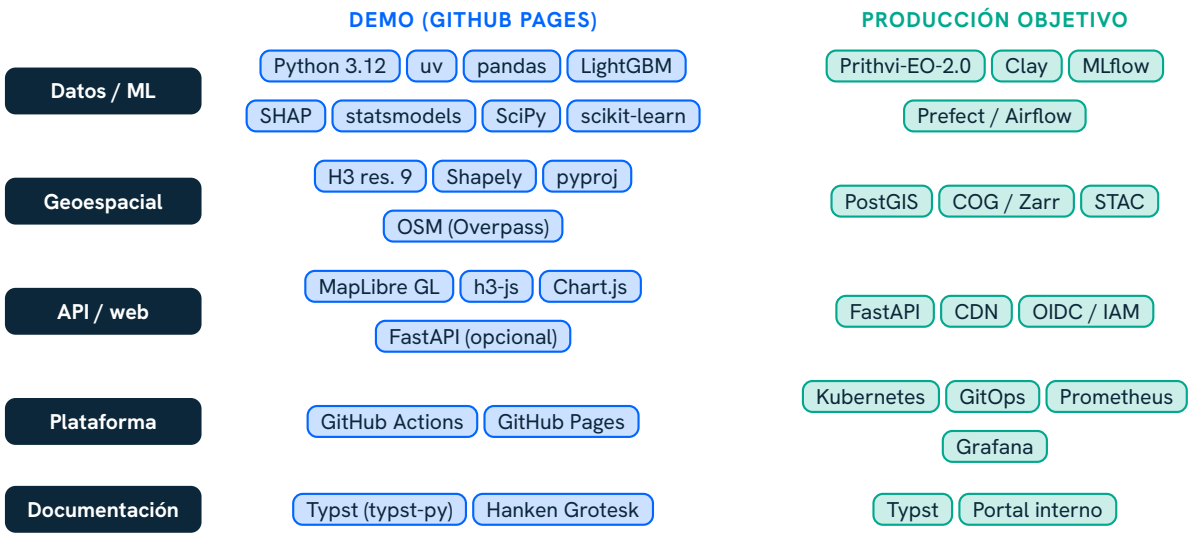


Figura 4: Stack por capa: demo (azul) y objetivo de producción (verde).

El stack de producción es una propuesta de referencia; la elección final (proveedor cloud, orquestador) depende del cliente y de los requisitos de soberanía del dato. ESTIMACIÓN

5.2 Justificación de elecciones clave

Tecnología	Por qué
Python 3.12 + uv	Ecosistema geoespacial y de ML; `uv` da entornos reproducibles y rápidos con `uv.lock`.
LightGBM + SHAP	Boosting eficaz con pocos datos tabulares; SHAP proporciona explicación local por celda.
statsmodels (STL) + SciPy (Theil-Sen)	Descomposición estacional y tendencia robusta a valores atípicos (nubes, huecos).
H3 res. 9	Malla jerárquica uniforme (≈ 0,1 km^2), agregable y fácil de teselar por ciudad.
MapLibre GL + Chart.js	Mapa vectorial y gráficos sin build step; versión fija por CDN.
Typst	Documentación reproducible desde texto, rápida y con control tipográfico.

6 Decisiones de arquitectura (ADR)

ADR	Decisión	Motivo y consecuencia
ADR-01	**Static-first** para GitHub Pages.	Pages sólo sirve ficheros estáticos; el pipeline corre offline y publica JSON. Coste de mantenimiento mínimo; la API pasa a ser opcional.
ADR-02	**H3 resolución 9** como unidad de análisis.	Equilibrio entre detalle de manzana y volumen de datos; permite escalar por tiles H3 a más ciudades.
ADR-03	**Umbrales aplicados en el cliente.**	El modelo entrega scores continuos; el técnico mueve el slider y la vista se recalcula al instante, sin servidor. Los umbrales vigentes se guardan en cada decisión.
ADR-04	Adaptador **DecisionStore** con dos implementaciones.	`LocalDecisionStore` (localStorage + exportación JSON) en Pages; `ApiDecisionStore` en producción. Misma interfaz, sin cambios en la UI.
ADR-05	Rutas **relativas** y sin dependencias de `localhost` .	Funciona bajo `usuario.github.io/repo/` .
ADR-06	Datos sintéticos **deterministas** con semilla.	Reproducibilidad y pruebas estables; claramente etiquetados como sintéticos.

7 Principios Clean Code aplicados

- **Responsabilidad única** y funciones pequeñas con nombres de dominio (`classify_cell`, `rank_proposals`).
- **Inversión de dependencias**: el dominio define puertos (`DataSource`, `DecisionStore`); la infraestructura los implementa.
- **Sin efectos laterales en el dominio**: funciones puras, entidades inmutables y tipado estático (Pydantic, `dataclass(frozen=True)`).
- **Configuración explícita** (`config.py`, `config.json`) en lugar de constantes mágicas; umbrales con valores por defecto documentados.
- **Errores explícitos y trazabilidad**: cada artefacto lleva `model_version`, `generated_at` y `schema_version`.
- **Linter y formato**: `ruff`; revisión por pares y CI obligatoria.

8 Estrategia de testing

Nivel	Qué se prueba	Herramienta
Unitario (dominio)	Clasificación por umbrales, normalización de scores, reglas de priorización.	pytest
Casos de uso	Pipeline con fuentes sintéticas y semillas; contrato de salida (esquema).	pytest
Contrato de datos	Validación de `manifest.json`, `cells.geojson`, etc. frente al esquema versionado.	pytest + Pydantic
API	`GET/POST /api/decisions` y esquema de registros.	httpx / TestClient
Modelo	AUC, F1 en el umbral crítico, MAE de previsión; comprobación de drift.	pytest + métricas
Web (manual / E2E)	Mapa con capas H3, umbrales, aprobar/rechazar, exportación, consola sin errores; ruta `/repo/` simulada.	navegador + servidor estático

HISTORIAL DE VERSIONES

Versión	Fecha	Cambios
0.1	25-10-2026	Estructura prevista en el plan.
0.2	25-10-2026	Arquitectura hexagonal, componentes, flujo, stack, ADR, Clean Code y testing alineados con el contrato de datos v1.